

Introduction to Programming Environment

- We want to solve problem with the help of computer system
- We need to interact with computer, so that we can give instructions to computer to perform the given task.
- We need to give instructions in a language so that computer can understand it.
- Computer understands only binary language (language in form of 0's and 1's only)
- We give some set/sequence of instructions in computer understandable form to perform a task.

The set/sequence of instructions to perform a specific task is called “Program”.

Basics of Programming

***We need a language to interact with computer**

Programming languages are classified as :

1. Machine language
- 2 . Assembly language
- 3 . High level language

Basics of Programming

MACHINE LANGUAGE :-

- the language of 0s and 1s
- system dependent
- different set of binary instructions for different types of computer systems .

LIMITATIONS :

- It is very tedious and error prone process of writing programs in machine languages.

Basics of Programming

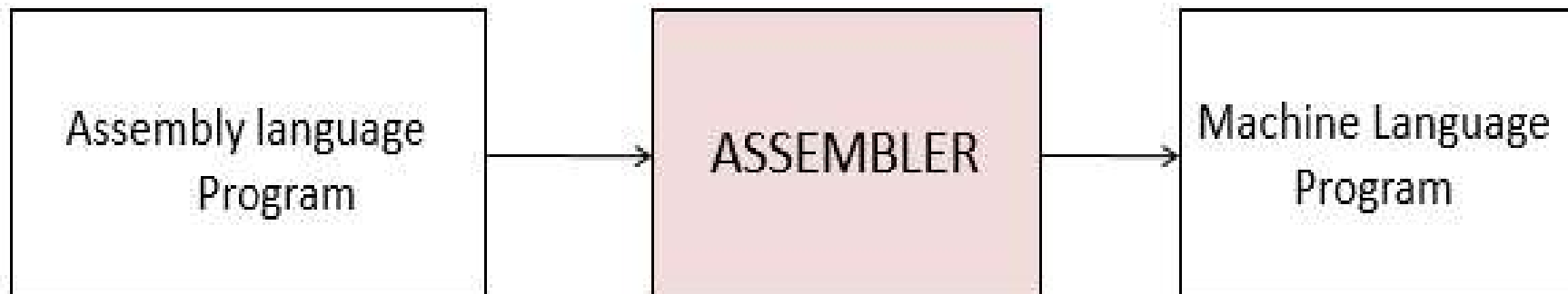
ASSEMBLY LANGUAGE:

- low level programming language
- the sequence of 0s and 1s are replaced by **mnemonic** (ni-monic) codes.

Example :- ADD for addition, SUB for subtraction etc

- system dependent program

* **Assembler(a system program)** is used to translate an assembly language program into the machine language program.



Example: an assembly language program

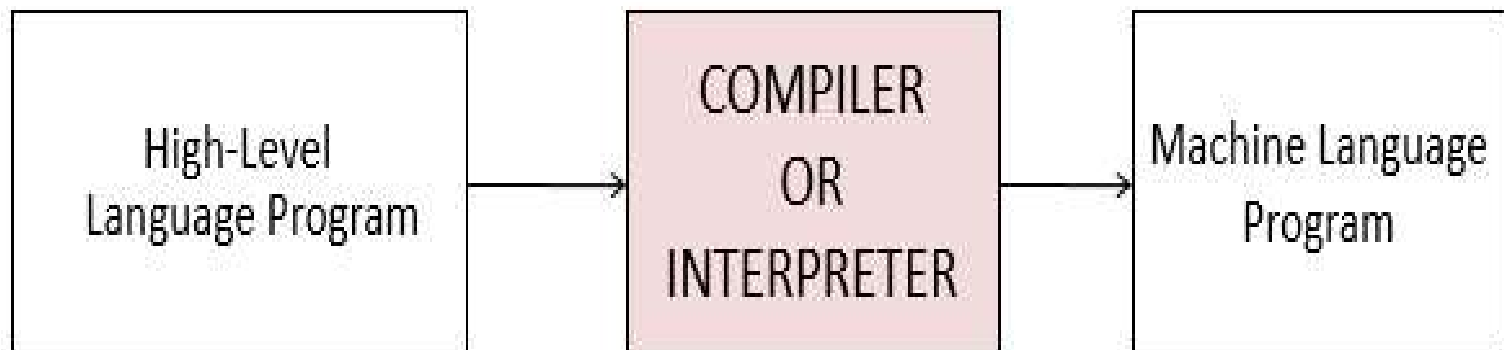
MVI C, 00	'Initialize C register to 00
LDA 4150	'Load the value to Accumulator.
MOV B, A	'Move the content of Accumulator to B register.
LDA 4151	'Load the value to Accumulator.
ADD B	'Add the value of register B to A
JNC LOOP	'Jump on no carry.
INR C	'Increment value of register C
LOOP: STA 4152	'Store the value of Accumulator (SUM).
MOV A, C	'Move content of register C to Acc.
STA 4153	'Store the value of Accumulator (CARRY)
HLT	'Halt the program.

Basics of Programming

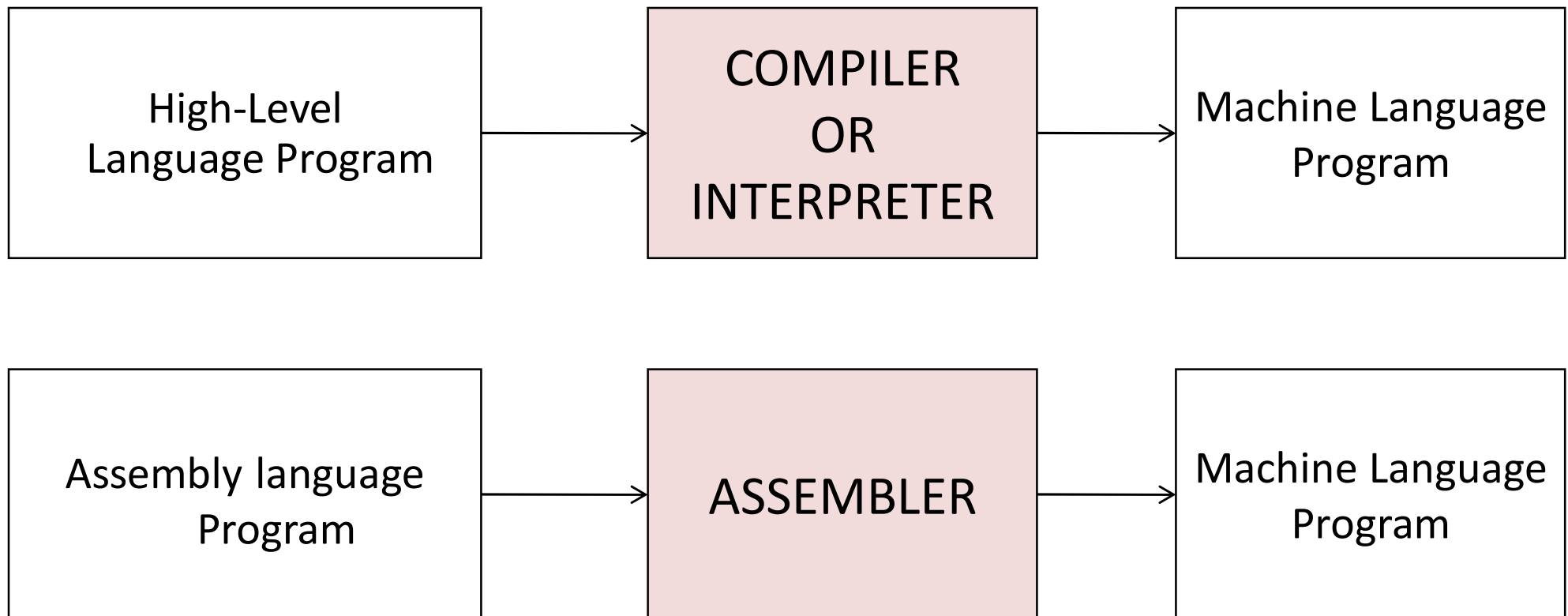
HIGH LEVEL LANGUAGE :-

- English like statements and programs
- programs are needed to be translated into Machine language
- **Compiler or Interpreter** are used for translation
- **programs are much easier to write, maintain and modified .**

***HIGH LEVEL LANGUAGE PROGRAM IS ALSO CALLED SOURCE CODE.**



Translators

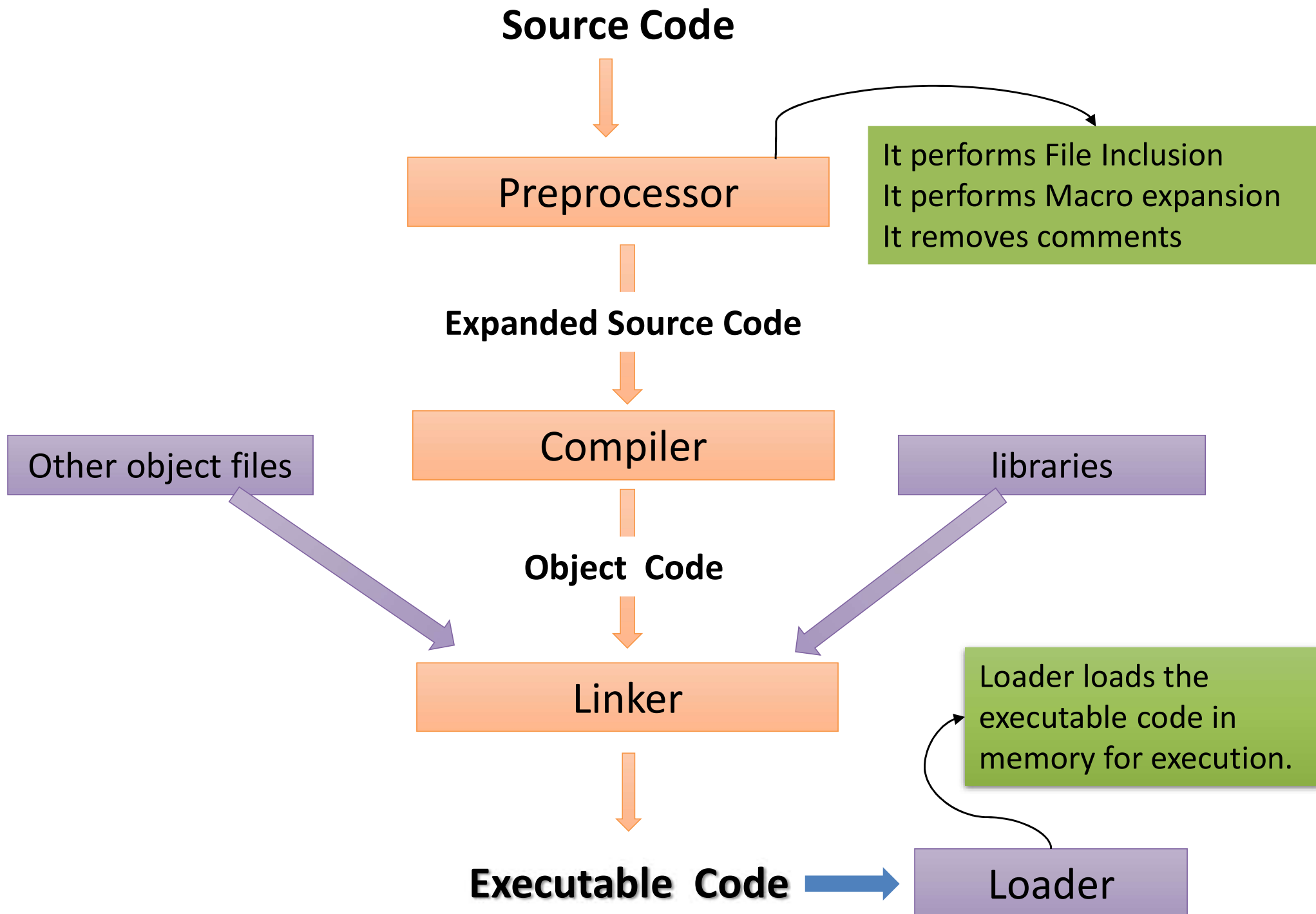


Linker

- A linker takes several object files and libraries as input and produces one executable object file.
 - It is a program that takes one or more objects generated by compiler and combines them into a single executable files.
 - **"Linking is simply the process of placing the address of a called function into the calling function's code. This is a fundamental software concept."**
-

Loader

- The loader is a component of the operating system responsible for loading the executable program into memory and preparing it for execution.
- This refers to copying a program image from hard disk to the main memory in order to put the program in a ready-to-run state (preparing program for execution).



Basics of C Programming

Father OF C



Dennis Ritchie
(1941-2011)

Introduced C in 1972

Features of C Programming Language

C is a procedural programming language. It was initially developed by **Dennis Ritchie in the year 1972**. It was mainly developed as a system programming language to write Unix operating system.

Here are some of the most important features of the C language:

1. Middle-Level Language including low-level access to memory
2. Procedural Language
3. Fast and Efficient
4. Modularity
5. General-Purpose Language (suitable for application programming and system programming like an operating system or compiler development both)
6. Libraries with Rich Functions

Structure of C Program

```
#include<stdio.h>
```

```
int main()
```

```
{
```

```
.....
```

```
.....
```

```
.....
```

```
.....
```

```
.....
```

```
.....
```

```
return 0;
```

```
}
```

```
#include<stdio.h>
```

```
int main()
```

```
{
```

```
instruction 1;
```

```
instruction 2;
```

```
instruction 3;
```

```
instruction 4;
```

```
.....
```

```
.....
```

```
return 0;
```

```
}
```

➤ **Every instruction must be terminated with semi-colon ;**

Structure of C Program

- **Identify variables:**

- number of inputs and their names
- number of outputs and their names
- intermediate variables(if needed) and their names

- **Types of variables:**

- integer
- floating point number
- characters
- strings(words)

- for integer we use **int**
- for floating point number we use **float**
- for character we use **char**

Structure of C program

Declaration of variables

```
int main()
```

```
{
```

```
    int A,B,C;
```

```
    .....
```

```
    .....
```

```
    .....
```

```
    return 0;
```

```
}
```

```
int main()
```

```
{
```

```
    float x, y;
```

```
    .....
```

```
    .....
```

```
    .....
```

```
    return 0;
```

```
}
```

```
int main()
```

```
{
```

```
    int num1,num2,C=90;
```

```
    float avg;
```

```
    .....
```

```
    .....
```

```
    .....
```

```
    return 0;
```

```
}
```

➤ ***Note: Every variable must be declared before use.***

Structure of C program

Every instruction may belong to either of 3 categories:

- *taking **input** from keyboard*
- *Calculation/Logic*
- *printing **output** to monitor*

Structure of C program

➤ Printing output to monitor:

we use a pre-defined function

printf()

printf() is used to print anything on monitor

Syntax :

printf(".....");

for example:

printf("PSIT");

will print **PSIT** on monitor.

Printing output in C: printf()

```
int main()
{
    printf("PSIT");
    printf(" Kanpur");
}
```

This program will print
PSIT Kanpur

```
int main()
{
    printf("Ramesh Kumar,");
    printf("\n 678, Govind Nagar,");
    printf("\nknapur-208011");
}
```

**Ramesh Kumar,
678, Govind Nagar,
Knapur-208011**

Printing output in C: printf()

What if we want to print the value of a variable?

```
int main()  
{  
    int X=10,y=20;  
    printf("X");  
    printf("y");  
    return 0;  
}
```

X y

But we want to print the value of X not X itself.
we want to print the value of y not y itself.

Printing output in C: printf()

we use **format-specifier** to print the value of a variable

For int **%d**

For float **%f**

For char **%c**

```
int main()
```

```
{
```

```
    int x=10;
```

```
    printf("%d", x);
```

```
    return 0;
```

```
}
```

10

Will print **10 (value of x)**

```
int main()
```

```
{
```

```
    int x=10;
```

```
    float y=2.5;
```

```
    printf("%d",x);
```

```
    printf(" ");
```

```
    printf("%f",y);
```

```
    return 0;
```

```
}
```

10 2.500000

Printing output in C: printf()

```
int main()
{
    int a,b;
    a=10;
    b=20;
    printf("a=%d", a);
    printf(" b=%d", b);
    return 0;
}
```

a=10 b=20

```
int main()
{
    int a,b;
    a=10;
    b=20;
    printf("value of a=%d",100);
    printf(" value of b=%d",b);
    return 0;
}
```

value of a=100 value of b=20

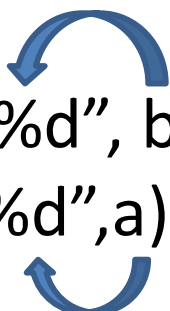
Printing output in C: printf()

- **%d** is the place holders for any variable
- In **printf()** function where **%d** is present, **%d** will be replaced by the value of integer provided.
- The same is true for **%f** and **%c** also.

Printing output in C: printf()

What will be printed???

```
int main()
{
    int a,b;
    a=10;
    b=20;
    printf(" a= %d", b);
    printf(" b=%d",a);
    return 0;
}
```



a=20 b=10

- Every thing inside **printf()** will be printed as it is.
- **%d** will be replaced by the value of integer provided

Printing output in C: printf()

➤ Important:

include header file at the top of program to refer pre-defined functions.

#include<stdio.h>

```
#include<stdio.h>
int main()
{
    int a,b;
    a=10;
    b=20;
    printf(" a= %d", b);
    printf(" b=%d",a);
    return 0;
}
```

a=20 b=10

Taking input from user via keyboard

To take input from user we use a pre-defined function:

scanf()

scanf() is used to take the input from user via **keyboard**.

Syntax:

```
int a,b;  
scanf("%d",&a);  
scanf("%d",&b);
```

Taking input from user via keyboard

scanf() is used to take the input from user via **keyboard**.

Syntax:

```
int main()
{
    int a,b;
    scanf("%d",&a);
    scanf("%d",&b);
    .....
    .....
}
```

```
int main()
{
    int a,b;
    scanf("%d %d",&a,&b);
    .....
    .....
}
```

```
int main()
{
    float a,b;
    scanf("%f",&a);
    scanf("%f",&b);
    .....
    .....
}
```

Important: Do not use any thing other than %d or %f inside double quotes of scanf() function.

Sample Program : Addition of two numbers

```
#include<stdio.h>
```

```
int main()
```

```
{
```

```
    int a,b,sum;
```

```
    scanf("%d",&a);
```

```
    scanf("%d",&b);
```

```
    sum=a+b;
```

```
    printf("sum=%d",sum);
```

```
    return 0;
```

```
}
```

```
#include<stdio.h>
```

```
int main()
```

```
{
```

```
    int a,b,sum;
```

```
    scanf("%d %d",&a,&b);
```

```
    sum=a+b;
```

```
    printf("sum=%d",sum);
```

```
    return 0;
```

```
}
```

```
5
```

```
7
```

```
sum=12
```

Preprocessor Directives in C

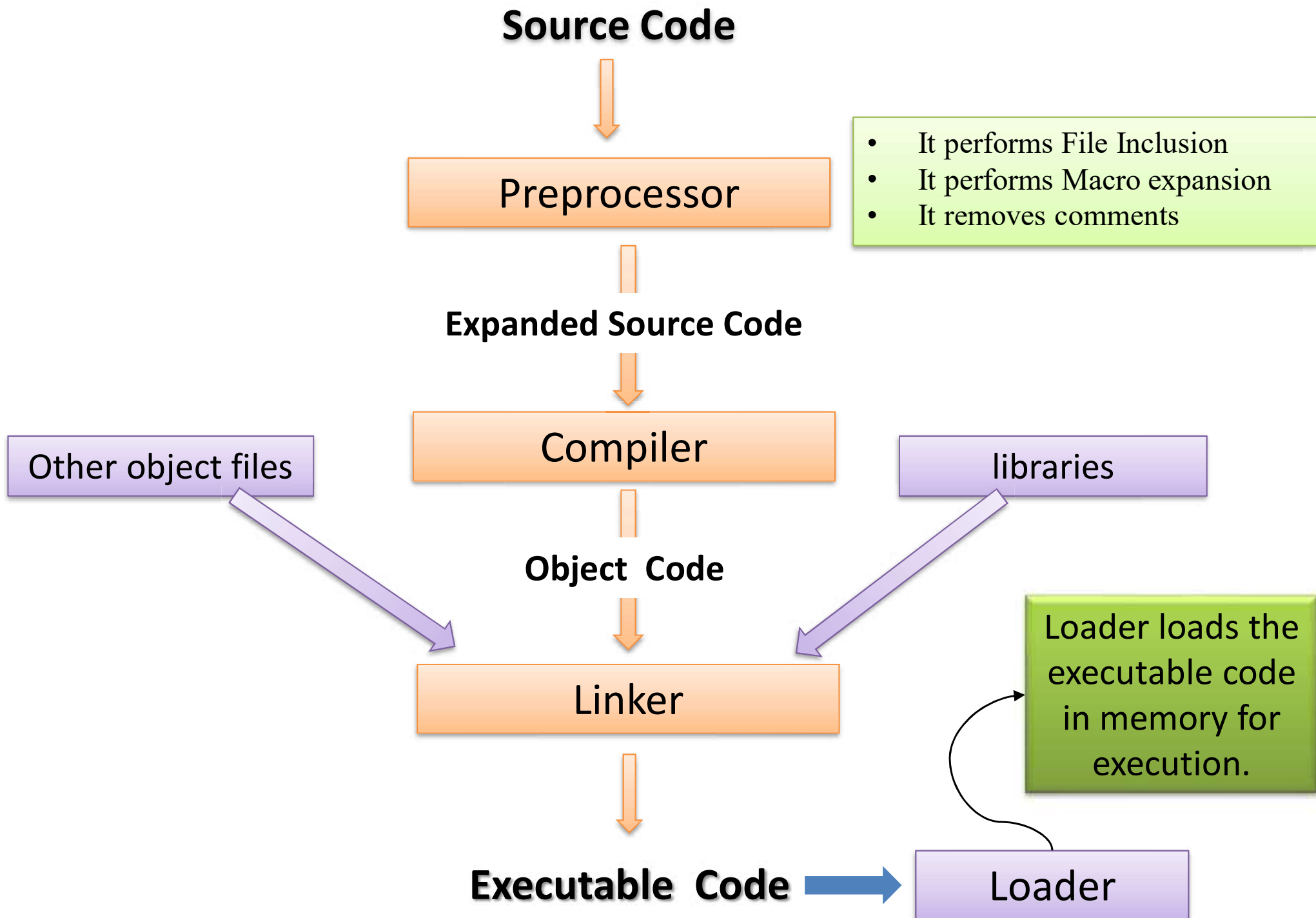
- Commands used in preprocessor are called preprocessor directives and they begin with “#” symbol.
- Preprocessor directives are executed before compilation.
- Examples:

`#include<stdio.h>`

`#include<math.h>`

`#define pi 3.1419`

- It performs File Inclusion
- It performs Macro expansion
- It removes comments



Basic Structure of C Program

Documentation Section
Link Section
Definition Section
Global Declaration Section
main() function Section { }
Subprogram Section function1 { } function2 { }

The structure of a C Program is divided into 6 sections:

1. Documentation Section
2. Link Section
3. Definition Section
4. Global Declaration Section
5. main() function Section
6. Sub-program Section

Basic Structure of C Program

Documentation Section
Link Section
Definition Section
Global Declaration Section
main() function Section { Declaration Part Executable Part }
Subprogram Section function1 { } function2 { }

Documentation Section

This section consists of comment lines.
Documentation section helps anyone to get an overview of the program.

// single line comment

/*..... multiline comments

.....

****/***

Link Section

The link section consists of the header files of the functions that are used in the program.

Basic Structure of C Program

Documentation Section
Link Section
Definition Section
Global Declaration Section
<pre>main() function Section { Declaration Part Executable Part }</pre>
<pre>Subprogram Section function1 { } function2 { }</pre>

Definition Section

- All the symbolic constants (macros) are written in definition section.

For example:

```
#define pi 3.1419
#define SQUARE(x) x*x
```

Global Declaration Section

- The global variables that can be used anywhere in the program are declared in global declaration section.
- This section also declares the user defined functions.

Basic Structure of C Program

Documentation Section
Link Section
Definition Section
Global Declaration Section
<pre>main() function Section { Declaration Part Executable Part }</pre>
<pre>Subprogram Section function1 { } function2 { }</pre>

main() Function Section:

It is necessary have one main() function section in every C program.

The execution of program starts at opening braces and ends at closing braces.

- It is called by Operating system
- It returns to Operating system
- It is also called driver function to start the execution of C program

Basic Structure of C Program

Documentation Section
Link Section
Definition Section
Global Declaration Section
main() function Section { Declaration Part Executable Part }
Subprogram Section function1 { } function2 { }

Subprogram Section

- It contains all the user defined functions that are used to perform a specific task.
- These user defined functions are called in the main() function directly or indirectly.

Documentation Section	{	// program to calculate
Link Section	{	#include<stdio.h> #include<math.h>
Definition Section	{	#define pi 3.1419 #define size 10
Global Declaration Section	{	int a; void fun1(); void fun2();
main() function Section	{	int main(void) { int x,y; printf("enter values"); }
Subprogram Section	{	void fun1() { } void fun2() { }